ALGOMANIACS 2026

Mathematics (Lecture 2)

-By Harsh

Binary Exponentiation

$$a^b = a^{b/2} * a^{b/2} * a^{b\%2}$$

This recursive formula means that we can calculate a^b in $O(\log_2 n)$

```
ll power(ll a,ll b){
    if(b==0) return 1;
    else if(b==1) return a;
    else{
        ll x=power(a,b/2);
        ll y=x*x;
        if(b%2!=0) return y*a;
        else return y;
    }
}
```

Modular Inverse

Fermat's Little Theorem :

$$a^{p-1} = 1 \pmod{p}$$

From this we can derive,

$$a^{-1} = a^{p-2} \pmod{p}$$

```
const ll mod = 1e9+7;

ll power(ll a,ll b){
    if(b==0) return 1;
    else if(b==1) return a;
    else{
        ll x=power(a,b/2);
        ll y=x*x;
        y%=mod;
        if(b%2!=0) return (y*a)%mod;
        else return y;
    }
}

ll modInverse(ll a,ll m){
    return power(a,m-2);
}
```

Combinatorics

```
const ll TOT=1e6;
vector<ll> factorial(TOT+1);
vector<ll> invfactorial(TOT+1);
void fac(){
    factorial[0]=1;
    for(ll i=1;i<=TOT;i++) factorial[i]=(factorial[i-1]*i) % mod;
}
void invfac(){
    invfactorial[TOT]=modInverse(factorial[TOT],mod);
    for(ll i=TOT;i>=1;i--) invfactorial[i-1]=(invfactorial[i]*i) % mod;
}
ll ncr(ll n,ll r){
    if(n<0 || r<0 || n<r) return -1;
    return ((factorial[n]*invfactorial[r] % mod)*invfactorial[n-r]) % mod;
}
```

In many questions, we need to find factorials of numbers modulo some big primes, e.g 1e9+7.

This code segment is used for this purpose as well as helps in calculating nCr.

https://codeforces.com/problemset/problem/300/C

Sieve of Eratosthenes

Using brute force, we can check if a number is prime in O(√n) time. Using Sieve of Eratosthenes, we can cut that down to constant time with a O(n(log(log n))) preprocessing step

```
int n;
vector<bool> is_prime(n+1, true);
is_prime[0] = is_prime[1] = false;
for (int i = 2; i <= n; i++) {
    if (is_prime[i] && (long long)i * i <= n) {
        for (int j = i * i; j <= n; j += i)
            is_prime[j] = false;
    }
}
```

By doing slight modifications in the normal code, we can also get all the primes up till n and the smallest prime factor of each number in the same time complexity.

```
const ll MAXN=1e6;
vector<ll> sieve(MAXN+1);
void sieve_of_eratosthenes(){
    for(ll i=2;i<=MAXN;i++) sieve[i] = i;
    for(ll i=4;i<=MAXN;i+=2) sieve[i] = 2;
    for(ll i=3;i*i<=MAXN;i++){
        if(sieve[i]==i){
            for(ll j=i*i;j<=MAXN;j+=i){
                if(sieve[j]==j) sieve[j] = i;
            }
        }
    }
}
```

Euclidean Algorithm

$$\gcd(a,b) = \begin{cases} a, & \text{if } b = 0 \\ \gcd(b, a \bmod b), & \text{otherwise.} \end{cases}$$

```
int gcd (int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd (b, a % b);
}
```

Time complexity of this algorithm is log(min(a,b)).

The built-in C++ function __gcd() works in the same manner.

Some Properties of Prime Divisors

Suppose prime factorisation of n is of the form $p_1^{e_1} \cdot p_2^{e_2} \cdots p_k^{e_k}$

Then,

Number of Divisors : $d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdots (e_k + 1)$

Sum of Divisors : $$\sigma(n) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1} \cdots \frac{p_k^{e_k+1} - 1}{p_k - 1}$$

Product of Divisors : $P(n) = n^{d(n)/2}$

Euler's Totient Function : Represents number of numbers < n coprime to n

$$\varphi(n) = \prod_{i=1}^{k} p_i^{\alpha_i - 1}(p_i - 1).$$

Problem 1

You are given a single integer n. Compute the **number of positive divisors of** $n!$. Since the answer can be very large, output it modulo $10^9 + 7$

Constraint : n <= 10^5

Hint : Use Legendre's formula

$$e_p = \left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \cdots$$

Problem 2

Given n compute :

$$\sum_{1 \leq i < j \leq n} \gcd(i, j)$$

Problem 3

Given n compute :

$$\sum_{1 \leq i < j \leq n} \operatorname{lcm}(i, j)$$

Möbius Function

The **Möbius function** $\mu(n)$ is defined as:

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1 \\ 0 & \text{if } n \text{ has a squared prime factor} \\ (-1)^k & \text{if } n \text{ is product of } k \text{ distinct primes} \end{cases}$$

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1 \\ 0 & n > 1 \end{cases}$$

$$[\gcd(a,b) = 1] = \sum_{d|\gcd(a,b)} \mu(d)$$

Problem 4

Given t test cases. For each test case, given n, output the **number of divisors of n**.

Constraints:

$$1 \leq t \leq 10^5$$
$$1 \leq n \leq 10^6$$

Nth Fibonacci in Log(N)

We use a method similar to binary exponentiation, just instead of a number we take a matrix.

Give {{1,1},{1,0}} as input to mpow() and the number at positon (0,1) in Nth power of the matrix is the Nth Fibonacci.

```
vector<vll> matmul(vector<vll> &a,vector<vll> &b){
    vector<vll> q(2,vll(2,0));
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            for(int k=0;k<2;k++){
                q[i][j]+=a[i][k]*b[k][j];
                q[i][j]=(q[i][j]%mod+mod)%mod;
            }
        }
    }
    return q;
}

vector<vll> mpow(vector<vll> x,ll n){
    if(n==0) return {{1,0},{0,1}};
    if(n==1){
        return x;
    }
    else{
        vector<vll> q=mpow(x,n/2);
        q = matmul(q,q);
        if(n%2==1) q = matmul(q,x);
        return q;
    }
}
```

Problem 5

You are given an $n \times n$ grid. Starting from the top-left cell (0,0), you need to reach the bottom-right cell (n-1,n-1). The maze is divided into two regions, with lava covering the upper-right half (i.e., all positions (i, j) where j > i). You can only move Right and Down. In how many different ways can he successfully cross the maze?

Constraints : $\mathrm{n} <= 10^6$

Catalan Numbers

C_n denotes the **nth Catalan number** – a specific number in the Catalan sequence.

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

Other places where C_n *can be used*:

- Number of **valid parentheses strings** with n pairs
- Number of paths from (0,0) to (n,n) that **never go above diagonal**
- Number of ways to correctly match n pairs
- Number of binary trees with n nodes

Additional Resources

Number theory other important topics –
https://cp-algorithms.com/algebra/extended-euclid-algorithm.html
https://cp-algorithms.com/algebra/linear-diophantine-equation.html
https://cp-algorithms.com/algebra/fibonacci-numbers.html
https://cp-algorithms.com/algebra/chinese-remainder-theorem.html
https://forthright48.com/category/cpps/number-theory/

PNC other important topics -
https://usaco.guide/gold/combo?lang=cpp#binomial-coefficients
https://cp-algorithms.com/combinatorics/burnside.html

Pseudo-Random Based Prime Fact –
https://cp-algorithms.com/algebra/ factorization.html#pollards-rho-algorithm

Info/Questions on Inclusion/Exclusion - https://codeforces.com/blog/entry/64625

Newton’s method tc - https://en.citizendium.org/wiki/Newton %27s_method#Computational_complexity

Maths other imp stuff - Probability, Matrices, FFT, Geometry(Very Rare)

Thank You